MODELOS DE LINGUAGEM E GENERATIVOS

Modelos de Linguagem e Generativos

Roteiro da Aula 2: Fundamentos da Representação Vetorial

1. Leis de Frequência

Estatística de texto com Luhn (1957) e a Lei de Zipf.

2. Ecossistema Python

Ferramentas essenciais: NLTK e Gensim para PLN.

3. Pré-Processamento

Tokenização, limpeza de Stopwords, Stemming e Corpus.

4. Modelos Esparsos

Bag-of-Words (BoW) e a ponderação estatística TF-IDF.

5. Word Embeddings

Representações densas com Word2Vec (CBOW e Skip-Gram).

6. Machine Learning

Busca por similaridade (não supervisionada) e classificação.

Módulo 1: H. P. Luhn e o Valor Informativo da Frequência (1957)

O Princípio Fundamental de Luhn

• Proposta Histórica (1957): O peso de um termo em um documento é proporcional à sua frequência de ocorrência (Term Frequency - TF).

• Pistas do Conteúdo: A frequência de palavras indica os assuntos principais do texto.

• O Problema dos Extremos:
  - Alta Frequência (artigos, preposições): baixíssimo valor discriminativo.
  - Baixa Frequência (palavras raríssimas, erros): podem gerar ruído estatístico.

Região de Alto Poder Resolutivo

• Corte Superior (Upper Cutoff):
  Elimina palavras gramaticais comuns (Stopwords) que não diferenciam documentos.

• Corte Inferior (Lower Cutoff):
  Elimina termos que ocorrem apenas 1 vez para economizar memória e evitar overfitting.

• Região Média (High Resolving Power):
  Palavras com frequência intermediária contêm o maior valor semântico para indexação.

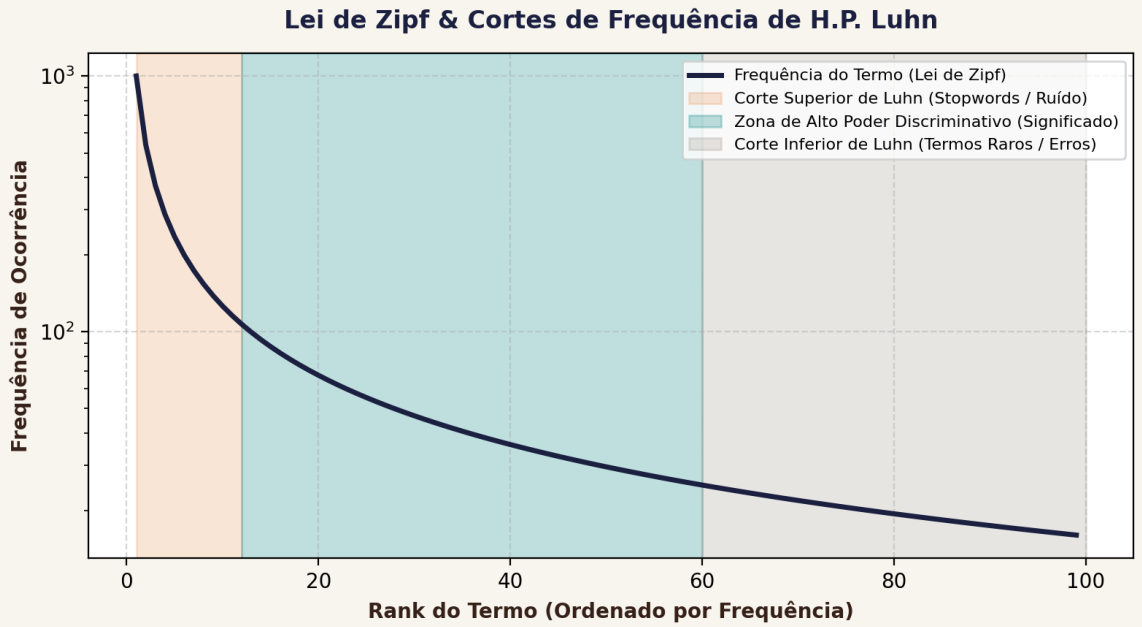
A Lei Universal de Zipf e a Distribuição de Frequência

A Formulação de George K. Zipf

• Lei Inversamente Proporcional:
A frequência f(n) de uma palavra é inversamente proporcional ao seu rank n:
$f(n) \propto 1 / n$

• Padrão Exponencial em Qualquer Língua:
A 1ª palavra mais comum aparece ~2x mais que a 2ª, ~3x mais que a 3ª.

• Evidência nos Notebooks da Aula:
- Em 'Dom Casmurro': 'que' (2.663), 'de' (1.975), 'me' (1.034), 'se' (857).
- Na Wikipédia ('Kate Beckinsale'): 'de' (231), 'em' (127), 'que' (88).



Acesse: Aula_02_Zipf_Law.ipynb

Módulo 2: O Ecossistema Python para PLN — NLTK e Gensim

NLTK (Natural Language Toolkit)

• Foco: Tarefas fundamentais de Processamento de Linguagem Natural.

• Principais Módulos Utilizados:
  - `nltk.tokenize`: `word_tokenize`, `sent_tokenize`.
  - `nltk.corpus`: Mapeamento de `stopwords` em português e corpora clássicos (`machado`).
  - `nltk.stem`: Radicalização com `RSLPStemmer`.

• Inicialização no Notebook:
  `import nltk`
  `nltk.download('punkt')`
  `nltk.download('stopwords')`

Gensim (Generate Similar)

• Foco: Modelagem de tópicos, representação vetorial e aprendizagem não supervisionada.

• Principais Módulos Utilizados:
  - `gensim.corpora.Dictionary`: Criação e gerenciamento de dicionários e vocabulário.
  - `gensim.models.TfidfModel`: Ponderação estatística TF-IDF.
  - `gensim.models.Word2Vec`: Arquiteturas CBOW e Skip-Gram.

• Inicialização no Notebook:
  `import gensim`
  `from gensim.models import Word2Vec`

Módulo 3: Preparação do Texto — Tokenização com NLTK

O Conceito de Tokenização

• Definição: Fracionar o texto em unidades discretas chamadas 'tokens'.

• Normalização de Caixa (Lowercasing): `doc.lower()` garante que termos em maiúsculas/minúsculas sejam unificados.

• Desafios de Limpeza: Pontuações grudadas, hashtags e links de internet exigem rotinas de limpeza.

```
# Código Python (NLTK / Gensim)
import nltk
from nltk.tokenize import word_tokenize

# Tokenizando corpus de documentos
doc_words = []
for doc in df.texto:
    tokens = word_tokenize(doc.lower())
    doc_words.append(tokens)
```

Saída do Notebook:

```
doc_words[0] = ['risco', 'com', 'mercado', 'seria',
 'maior', 'do', 'que', 'com', 'empresa', ',']
doc_words[1] = ['analise', 'de', 'dados', 'e',
 'modelos', 'de', 'linguagem', 'em', 'python']
```

Limpeza de Texto: Stop Words e Pontuação

Remoção de Ruído Semântico

- O que são Stop Words?
  Palavras gramaticais sem conteúdo semântico próprio (artigos, preposições, pronomes).

- Lista NLTK em Português:
  `stopwords.words('portuguese')` contém 200+ palavras como 'de', 'que', 'com', 'para', 'em'.

- Filtragem de Pontuação:
  Uso de `string.punctuation` para remover pontuações e símbolos.

```
# Código Python (NLTK / Gensim)
from nltk.corpus import stopwords
import string

stop_words = stopwords.words('portuguese')
trash = list(string.punctuation)

# Filtragem no notebook
clean_tokens = [w for w in tokens
                if w not in stop_words and w not in trash]
```

Saída do Notebook:

```
Antes: ['risco', 'com', 'mercado', 'seria', 'maior', 'do', 'que',
'com', 'empresa', ',']

Depois: ['risco', 'mercado', 'maior', 'empresa', 'diz', 'analista']
```

Normalização Morfológica: Stemming com NLTK

O Algoritmo RSLP (Stemming)

• Conceito de Stemming:
Remoção heurística de sufixos para extrair o radical morfológico bruto da palavra.

• RSLPStemmer no NLTK:
`RSLPStemmer` é um algoritmo baseado em regras para a língua portuguesa.

• Vantagens vs. Desvantagens:
- Vantagem: Reduz drasticamente a dimensão do vocabulário.
- Desvantagem: Trunca palavras de forma pouco inteligível para humanos.

```
# Código Python (NLTK / Gensim)
import nltk
nltk.download('rslp')
stemmer = nltk.stem.RSLPStemmer()

print(stemmer.stem('economista'))
print(stemmer.stem('economia'))
print(stemmer.stem('econômico'))
```

Saída do Notebook:

```
econom
econom
econôm

Entrada: ['processamento', 'linguagem', 'natural']
Saída:   ['process', 'linguag', 'natur']
```

Construção de Corpus e Dicionário Gensim

O Dicionário de Vocabulário

• O que é um Corpus em PLN?
  Coleção completa de todos os documentos limpos e tokenizados (1.000 documentos no exemplo).

• `gensim.corpora.Dictionary`:
  Mapeia cada palavra única do corpus para um ID numérico inteiro exclusivo.

• Redução do Vocabulário (Clean Dictionary):
  - Antes da limpeza: 4.255 tokens únicos.
  - Após limpeza (stopwords + pontuação): 4.133 tokens únicos.

```
# Código Python (NLTK / Gensim)
import gensim

# Criando o dicionário no Gensim
dictionary = gensim.corpora.Dictionary(doc_words)

print(dictionary.token2id)
print(len(dictionary.token2id))
```

Saída do Notebook:

```
{'empresa': 0, 'diz': 1, 'analista': 2, 'maior': 3,
 'mercado': 4, 'risco': 5, 'atrativo': 7, ...}

Tamanho total do dicionário: 4133 palavras únicas
```

Módulo 4: Bag of Words (BoW) — O Modelo Vetorial Esparso

A Intuição do Bag of Words

• Sacola de Palavras (BoW):
Representa um documento como contagem de frequência de suas palavras, descartando a ordem e a estrutura.

• Saída Gensim (`doc2bow`):
Formato de lista de tuplas `(ID_da_palavra, Frequência)`.

• Esparsidade Vetorial:
Vetor de tamanho igual ao vocabulário (4.133 posições), onde >99% das posições são zero.

```
# Código Python (NLTK / Gensim)
# Gerando BoW para o 1º documento
doc0 = ['risco', 'mercado', 'maior', 'empresa', 'diz', 'analista']
bow_doc0 = dictionary.doc2bow(doc0)
print(bow_doc0)
```

Saída do Notebook:

```
[(0, 1), (1, 1), (2, 1), (3, 1), (4, 1), (5, 1)]

Significado: O termo ID 0 ('empresa') ocorre 1 vez;
o termo ID 1 ('diz') ocorre 1 vez, etc.
```

Ponderação Estatística — O Modelo TF-IDF

Fórmula e Mecanismo do TF-IDF

• Problema do BoW Puro:
Palavras frequentes em todo o corpus continuam recebendo alto peso.

• TF (Term Frequency): Frequência do termo no documento local.

• IDF (Inverse Document Frequency):
Penaliza palavras comuns em muitos documentos:
IDF(t) = log(N / DF(t))

• Resultado: Destaca palavras raras e informativas.

```
# Código Python (NLTK / Gensim)
tf_idf_model = gensim.models.TfidfModel(corpus)
tfidf_doc0 = tf_idf_model[bow_doc0]
print(tfidf_doc0)
```

Saída do Notebook:

```
BoW   -> (2, 1) ['analista']
TFIDF -> (2, 0.6226)  <-- Peso alto por ser raro!

BoW   -> (0, 1) ['empresa']
TFIDF -> (0, 0.2254)  <-- Peso menor por ser comum!
```

Acesse: Aula_02_BOW_TFIDF_word2vec.ipynb

Módulo 5: Representações Densas — Word Embedding & Word2Vec

A Revolução do Word2Vec (2013)

• Salto Tecnológico (Mikolov et al., Google 2013): Substituição de vetores esparsos de 4.133D por vetores densos de 50D a 300D.

• Hipótese Distribucional (Firth, 1957): 'Você conhecerá a palavra pelas companhias que ela mantém.' Palavras com contextos similares possuem vetores próximos.

• Vantagens: Valores contínuos (floats), operações algébricas e eliminação de esparsidade.

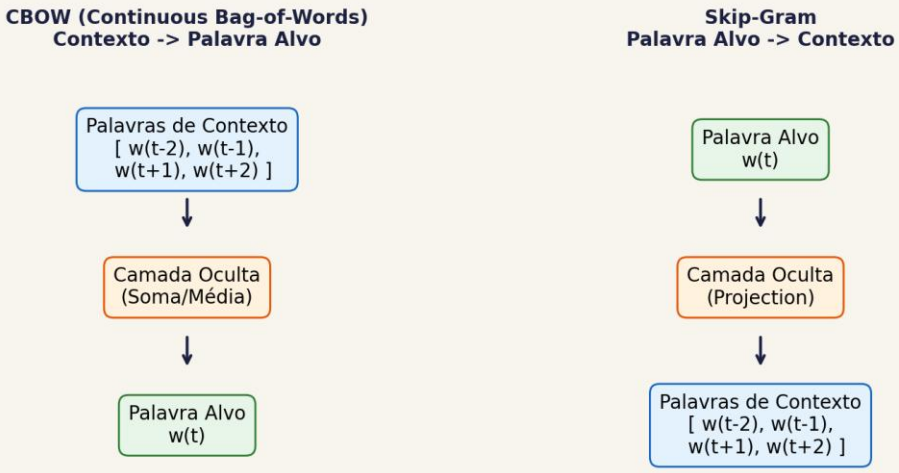


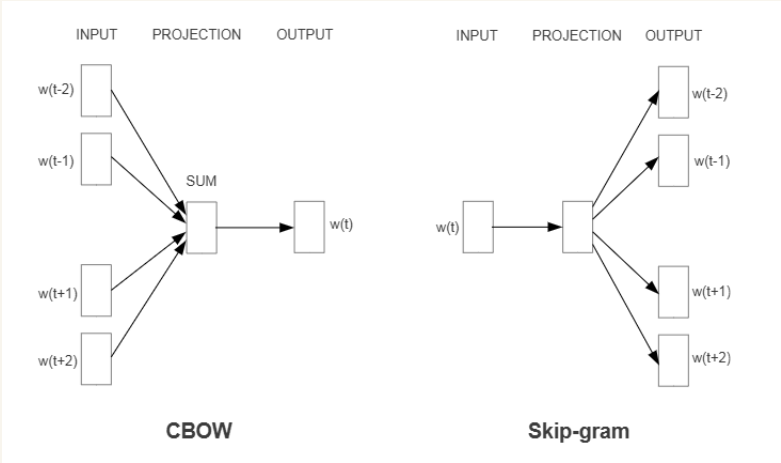
Arquiteturas do Word2Vec: CBOW vs. Skip-Gram

Comparativo das Arquiteturas

• CBOW (Continuous Bag-of-Words):
  - Prevê a palavra central w(t) a partir das palavras de contexto ao redor.
  - Gensim: `sg = 0` (`vector_size=50, window=5`)
  - Treinamento rápido, ideal para palavras frequentes.

• Skip-Gram:
  - Prevê o contexto a partir da palavra central w(t).
  - Gensim: `sg = 1`
  - Excelente para corpora menores e captura de palavras raras.



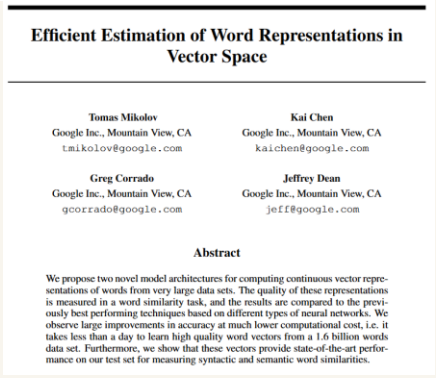


Arquiteturas do Word2Vec: CBOW vs. Skip-Gram

1. **Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013).** Efficient Estimation of Word Representations in Vector Space. arXiv.

Table 1: *Examples of five types of semantic and nine types of syntactic questions in the Semantic-Syntactic Word Relationship test set.*

Type of relationship	Word Pair 1		Word Pair 2	
Common capital city	Athens	Greece	Oslo	Norway
All capital cities	Astana	Kazakhstan	Harare	Zimbabwe
Currency	Angola	kwanza	Iran	rial
City-in-state	Chicago	Illinois	Stockton	California
Man-Woman	brother	sister	grandson	granddaughter
Adjective to adverb	apparent	apparently	rapid	rapidly
Opposite	possibly	impossibly	ethical	unethical
Comparative	great	greater	tough	tougher
Superlative	easy	easiest	lucky	luckiest
Present Participle	think	thinking	read	reading
Nationality adjective	Switzerland	Swiss	Cambodia	Cambodian
Past tense	walking	walked	swimming	swam
Plural nouns	mouse	mice	dollar	dollars
Plural verbs	work	works	speak	speaks



Efficient Estimation of Word Representations in Vector Space

Tomas Mikolov
Google Inc., Mountain View, CA
tmikolov@google.com

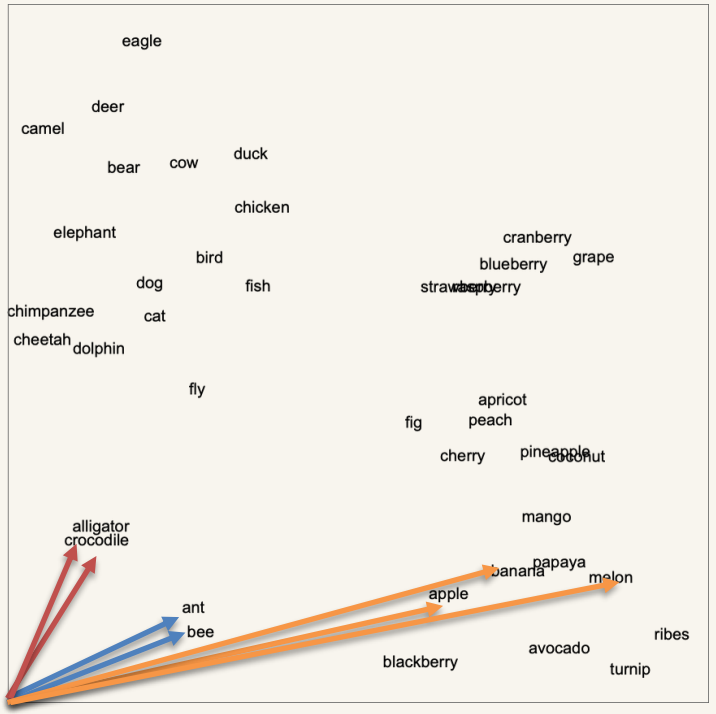
Kai Chen
Google Inc., Mountain View, CA
kaichen@google.com

Greg Corrado
Google Inc., Mountain View, CA
gcorrado@google.com

Jeffrey Dean
Google Inc., Mountain View, CA
jeff@google.com

Abstract

We propose two novel model architectures for computing continuous vector representations of words from very large data sets. The quality of these representations is measured in a word similarity task, and the results are compared to the previously best performing techniques based on different types of neural networks. We observe large improvements in accuracy at much lower computational cost, i.e. it takes less than a day to learn high quality word vectors from a 1.6 billion words data set. Furthermore, we show that these vectors provide state-of-the-art performance on our test set for measuring syntactic and semantic word similarities.



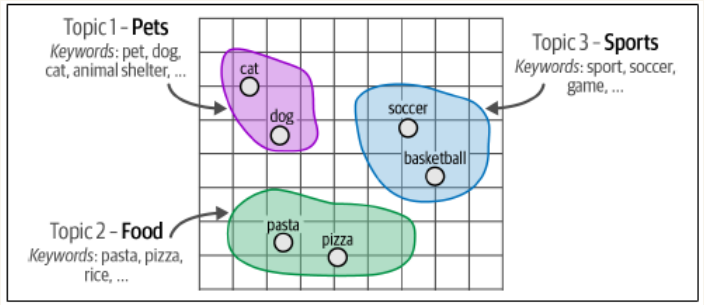


Figure 5-2. Topic modeling is a way to give meaning to clusters of textual documents.

Inspecionando Embeddings com Gensim

Explorando os Vetores de Palavras

- Vetor Denso da Palavra:
  Acessado via `model.wv['palavra']`. Retorna uma matriz NumPy de 50 números decimais.

- Busca por Similaridade (`most_similar`):
  Calcula a similaridade de cosseno no espaço vetorial entre o vetor e todos os termos do dicionário.

- Exemplo em Dom Casmurro:
  Palavras associadas a 'amor' no vocabulário de Machado de Assis.

```
# Código Python (NLTK / Gensim)
model_CBOW.wv['infantil'] # Vetor 50D

# Similaridade no corpus de Machado de Assis
model_SGRAM.wv.most_similar('amor', topn=5)
```

Saída do Notebook:

```
array([-0.01708, 0.01088, -0.01219, ..., 0.00197])

most_similar('amor'):
[('teria', 0.9971), ('amiga', 0.9970),
 ('muitas', 0.9970), ('passo', 0.9967)]
```

Document Embedding — Agregando Vetores de Palavras

Representação de Documentos Inteiros

• Como representar um documento com Word2Vec?
O Word2Vec gera vetores para palavras. Para o documento inteiro, calcula-se a média vetorial de suas palavras.

• Média dos Vetores de Palavras (Average Vector):
Soma-se os vetores de todas as palavras válidas do documento e divide-se pelo total de palavras.

• Saída Final:
Cada documento vira um único vetor denso de dimensão 50 (matriz 1.000 x 50 no notebook).

```
# Código Python (NLTK / Gensim)
doc_embed = []
for doc in doc_words:
    d = np.zeros(50)
    nr_words = 0
    for word in doc:
        if word in model_CBOW.wv.index_to_key:
            d += model_CBOW.wv[word]
            nr_words += 1
    if nr_words != 0: d /= nr_words
    doc_embed.append(d)
```

Saída do Notebook:

```
Tamanho da matriz de Document Embeddings:
1000 documentos x 50 dimensões densas
```

Módulo 6: Aprendizado Não Supervisionado — Busca por Similaridade

Busca por Vizinhos Mais Próximos

- Métrica de Cosseno (Cosine Distance):
  Mede o ângulo entre os vetores de documentos no espaço 50D, focando no conteúdo semântico.

- Consulta de Exemplo no Notebook:
  Query: 'segurança publica' -> gerado vetor de embedding da consulta.

- Resultados no Corpus:
  Recupera notícias sobre atuação policial e segurança, mesmo sem casar palavras exatas!

```
# Código Python (NLTK / Gensim)
from sklearn.neighbors import NearestNeighbors

nbrs = NearestNeighbors(n_neighbors=5, metric='cosine')
nbrs.fit(doc_embed)

distances, indices = nbrs.kneighbors(query_embed)
```

Saída do Notebook:

```
Query: 'segurança publica'
1. (dist 0.53) 'Polícia Civil apreende metralhadora...'
2. (dist 0.57) 'Polícia encontra menina chinesa...'
3. (dist 0.60) 'Polícia investiga caso em SP...'
```

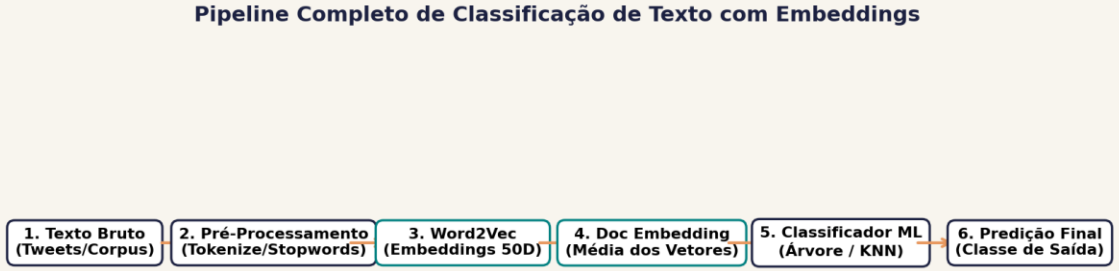
Módulo 7: Aprendizado Supervisionado — Classificação com Embeddings

Classificação Supervisionada

• Tarefa Prática do Notebook:
  Classificar a fonte/veículo de um documento com base em seu vetor de embedding.

• Configuração do Modelo:
  - Entradas X: Matriz de Document Embeddings (1000 x 50).
  - Rótulos y: Booleano do veículo de imprensa.
  - Divisão Treino/Teste: 70% treino, 30% teste com estratificação.

• Resultado no Notebook:
  Acurácia do modelo no teste: **80%** (`accuracy: 0.80`).



Estudo de Caso Prático: Corpus de Machado de Assis e NLTK

Análise de Estilo Literário

• Corpus Machado do NLTK:
`nltk.download('machado')` fornece 246 arquivos de romances, contos, teatro e crítica literária.

• Treinamento do Modelo em Dom Casmurro:
Vocabulário de 9.717 palavras únicas treinado com Skip-Gram (vector_size=50, window=8).

• Comparando 4 Textos Distintos:
- Textos 1 e 2: Machado de Assis (Dom Casmurro / O Alienista).
- Texto 3: Clarice Lispector.
- Texto 4: Wikipédia (Brasil).
Resultado: Textos do mesmo autor apresentam a maior proximidade vetorial!

```
# Código Python (NLTK / Gensim)
import nltk
nltk.download('machado')
from nltk.corpus import machado

doc_words = [list(machado.words('romance/marm08.txt'))]
model_SGRAM = Word2Vec(doc_words, vector_size=50, sg=1)
print(len(model_SGRAM.wv.index_to_key))
```

Saída do Notebook:

```
Total de termos únicos no romance: 9.717

Similaridade de Cosseno:
Machado vs Machado:  0.88 (Alta)
Machado vs Clarice:  0.64 (Média)
Machado vs Wikipédia: 0.42 (Baixa)
```

Acesse: Aula_02_BOW_TFIDF_word2vec.ipynb

Quadro Comparativo: Evolução das Representações Vetoriais

Critério	Bag of Words (BoW)	TF-IDF	Word2Vec (Static)	Transformers (LLMs)
Dimensão	Esparsa (N = Vocab)	Esparsa (N = Vocab)	Densa (50-300D)	Densa (768-12kD)
Frequência	Contagem Bruta	Ajustada por IDF	Implícita no treino	Atenção contextual
Semântica	Inexistente (Zeros)	Inexistente (Zeros)	Alta (Dist. Cosseno)	Contextual Dinâmica
Polissemia	Não trata	Não trata	Vetor Único (Limitação)	Diferenciada por frase

Leituras Recomendadas

1. **Hands-On Large Language Models:** Alammar, Jay; Grootendorst, Maarten (2024). O'Reilly Media. Capítulos 1 e 5.
2. **Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013).** Efficient Estimation of Word Representations in Vector Space. arXiv.
3. **NLTK and Gensim Documentation,** https://tedboy.github.io/nlps/